llnl_c1/lulesh · lulesh.cc · llm_pragmas_v5 · claude-haiku-4-5-20251001 · BROKEN · RIGHT = attempt 11 of 91, rejected by the gate at 'tsan'

−4 / +60 lines of 1967 · 7 parallel region(s) added · 30 of 91 candidate(s) kept · RESTRUCTURING (41 code line(s) added, 4 removed) · restructuring level 0

BEFORE		ATTEMPT 11 OF 91, REJECTED BY THE GATE AT 'TSAN'	
826	`const Real_t delt = domain.deltatime() ;`	826	`const Real_t delt = domain.deltatime() ;`
827	`Real_t u_cut = domain.u_cut() ;`	827	`Real_t u_cut = domain.u_cut() ;`
828		828	
829	`CalcForceForNodes(domain);`	829	`CalcForceForNodes(domain);`
830		830	
831	`CalcAccelerationForNodes(domain, domain.numNode());`	831	`Index_t numNode = domain.numNode();`
		832	`Index_t size = domain.sizeX();`
		833	`Index_t numNodeBC = (size+1)*(size+1);`
832		834	
833	`ApplyAccelerationBoundaryConditionsForNodes(domain);`	835	`// Combine remaining loops in a single parallel region to reduce synchroniz`
		836	`#pragma omp parallel`
		837	`{`
		838	`// CalcAccelerationForNodes - compute accelerations from forces`
		839	`#pragma omp for schedule(static) nowait`
		840	`for (Index_t i = 0; i < numNode; ++i) {`
		841	`domain.xdd(i) = domain.fx(i) / domain.nodalMass(i);`
		842	`domain.ydd(i) = domain.fy(i) / domain.nodalMass(i);`
		843	`domain.zdd(i) = domain.fz(i) / domain.nodalMass(i);`
		844	`}`
834		845	
835	`CalcVelocityForNodes( domain, delt, u_cut, domain.numNode()) ;`	846	`// ApplyAccelerationBoundaryConditionsForNodes - apply boundary conditio`
		847	`// Each symmetric plane is independent`
		848	`if (!domain.symmXempty() != 0) {`
		849	`#pragma omp for schedule(static) nowait`
		850	`for(Index_t i=0 ; i<numNodeBC ; ++i)`
		851	`domain.xdd(domain.symmX(i)) = Real_t(0.0) ;`
		852	`}`
836		853	
837	`CalcPositionForNodes( domain, delt, domain.numNode() );`	854	`if (!domain.symmYempty() != 0) {`
		855	`#pragma omp for schedule(static) nowait`
		856	`for(Index_t i=0 ; i<numNodeBC ; ++i)`
		857	`domain.ydd(domain.symmY(i)) = Real_t(0.0) ;`
		858	`}`
		859	
		860	`if (!domain.symmZempty() != 0) {`
		861	`#pragma omp for schedule(static) nowait`
		862	`for(Index_t i=0 ; i<numNodeBC ; ++i)`
		863	`domain.zdd(domain.symmZ(i)) = Real_t(0.0) ;`
		864	`}`
		865	
		866	`// CalcVelocityForNodes - update velocities from accelerations`
		867	`#pragma omp for schedule(static) nowait`
		868	`for ( Index_t i = 0 ; i < numNode ; ++i )`
		869	`{`
		870	`Real_t xdtmp, ydtmp, zdtmp ;`
		871	
		872	`xdtmp = domain.xd(i) + domain.xdd(i) * delt ;`
		873	`if( FABS(xdtmp) < u_cut ) xdtmp = Real_t(0.0);`
		874	`domain.xd(i) = xdtmp ;`
		875	
		876	`ydtmp = domain.yd(i) + domain.ydd(i) * delt ;`
		877	`if( FABS(ydtmp) < u_cut ) ydtmp = Real_t(0.0);`
		878	`domain.yd(i) = ydtmp ;`
		879	
		880	`zdtmp = domain.zd(i) + domain.zdd(i) * delt ;`
		881	`if( FABS(zdtmp) < u_cut ) zdtmp = Real_t(0.0);`
		882	`domain.zd(i) = zdtmp ;`
		883	`}`
		884	
		885	`// CalcPositionForNodes - update positions from velocities`
		886	`#pragma omp for schedule(static)`
		887	`for ( Index_t i = 0 ; i < numNode ; ++i )`
		888	`{`
		889	`domain.x(i) += domain.xd(i) * delt ;`
		890	`domain.y(i) += domain.yd(i) * delt ;`
		891	`domain.z(i) += domain.zd(i) * delt ;`
		892	`}`
		893	`}`
838		894	
839	`return;`	895	`return;`
840	`}`	896	`}`
841		897	
842	`static inline`	898	`static inline`

red = removed · green = added · yellow = parallel region (#pragma omp) · right strip = the whole file: green where changed, yellow where a parallel region now is